

Product Requirements Document — Buena Context Engine MVP (v2.0)

"We treat Slack, Emails, PDFs, and now Voice Calls simply as input sources that feed into our living Markdown file. But the Operational Human — the true property manager — needs a dedicated command center to control their AI agents. That is why we built this WebUI. It allows humans to curate the context, while the AI handles the execution — and the system gets smarter with every approved decision."

Hackathon target: Buena — The Context Engine (€2,500) **Stretch tracks (same submission, dual narrative):** telli & ai-coustics — Voice AI in the Wild **Timeline:** Hackathon duration (~24-48h) **Document version:** 2.0 — Voice + Learning Loop integrated **Last updated:** 2026-04-25

1. Problem & Opportunity

Property managers at Buena operate with fragmented knowledge spread across ERPs, emails, PDFs, Slack, voice calls, and their own heads. When a tenant ticket arrives — whether by email, chat, or a phone call about a broken radiator — the quality of the response depends on whether the human happens to remember the right context. This does not scale, and it is the exact reason "90% of property managers just react."

The opportunity: give every property a single **living, versioned, human-curated context document** that any automated helper can read before acting, that every human action can update, and that **gets smarter over time** through an asynchronous learning loop. This turns tribal knowledge into structured, auditable, evolving property memory — and accepts inputs from every channel a tenant might actually use, including noisy real-world phone calls.

2. Goal of the MVP

Ship a working demo that proves **four claims** in under 5 minutes:

1. **Every property has a living** `context.md` **that is versioned, diffable, and human-editable.**
2. **Every automated proposal is grounded** in a specific, auditable snapshot of that context and bounded by human-curated policies.
3. **Every human decision feeds back** into the context, so the system demonstrably improves per property.
4. **Tenants can reach the system through any channel** — including a noisy phone call from a real-world environment — and the proposal pipeline treats all inputs identically.

Success criteria for the demo

#	Criterion	How we show it
1	A ticket can be created via voice call, simulation, or API	Three ingestion paths visible in inbox
2	Voice input survives noise (street, café, kids in background)	Live demo with background noise playing; transcript still clean
3	A proposal is generated, tied to a context version, and cites sections	Proposal card shows <code>context v11</code> + clickable citations
4	An operator can approve, edit, or reject from the WebUI	Three buttons, each produces an audit entry
5	Approving a proposal creates a new context version	Version counter increments, diff is viewable
6	Editing a policy changes system behavior on the next ticket	Before/after same ticket type, different outcome
7	Slack receives a lightweight notification with a deep link	Message appears in channel, link opens the ticket
8	Hermes (learning loop) surfaces a context suggestion based on history	Suggestions panel shows pattern-derived recommendation
9	Every action is auditable	Audit page shows a timeline of reads, writes, decisions, learnings

Explicit non-goals (for the hackathon)

- Multi-user authentication or role-based access control
 - Real integrations with ERPs, email servers, or PDF ingestion at scale
 - Production-grade error handling, retries, rate limiting
 - Mobile responsiveness beyond "it doesn't break"
 - Localization framework (German seed content is fine; no i18n)
 - Outbound voice agents (the system answers calls, it does not place them)
-

3. User & Scenarios

Primary user: the Operator (Buena property manager)

Persona — Maria, 42, Hausverwalterin in Berlin-Mitte Manages 8 condominiums, lives in her inbox and Slack, trusts nothing she can't verify. Speaks German with tenants, English with Buena's tooling. Gets ~30 tenant interactions a day across phone, WhatsApp, email, and the front desk. Her biggest pain isn't volume — it's that she has to be the memory of every building. When she's on vacation, her colleague Ben re-asks every tenant the same questions because there's no shared context.

Secondary user: the Tenant (the person on the other end of the call)

Persona — Herr Schmidt, 68, Mieter in Gartenstraße 42 Calls the property manager from his living room while the TV is on. Doesn't use email reliably. Wants to report that his heating is cold, doesn't know the unit number off the top of his head, and gets frustrated when transferred. Today, he leaves a voicemail that Maria listens to three hours later. With our system, his call becomes a structured ticket within seconds, and Maria sees a proposal grounded in everything we already know about Gartenstraße 42 and Herr Schmidt's prior issues.

User stories

As Maria, the Operator, I want to...

1. ...see every tenant interaction — voice, email, Slack, manual entry — as a normalized ticket in one inbox, so I never miss a channel.
2. ...trust that any AI proposal is grounded in a specific version of the property's context, so I can verify the reasoning before approving.
3. ...edit the property's `context.md` directly when I learn something new (e.g., "the heating system was replaced in 2023"), so future proposals benefit from that knowledge.
4. ...change a policy (e.g., "auto-approve repairs under €200") and see it take effect on the next ticket, with a full audit trail.
5. ...receive a Slack notification when a proposal needs my decision, with a deep link straight to the ticket.
6. ...review Hermes' suggestions periodically (e.g., "you've approved 4 heating tickets under €200 this week — want to make this a policy?") and accept or dismiss them with one click.
7. ...see a clean diff of what changed in a property's context between versions, so I always know what the system "knows" and when it learned it.

As Herr Schmidt, the Tenant, I want to...

8. ...call the property manager and have my issue understood the first time, even if I'm calling from a noisy room or speaking quickly.

9. ...not have to repeat my building, my unit, or my prior issues — the system should already know me.
10. ...receive a callback or written confirmation that my issue was logged and is being handled, ideally within minutes.

As Ben, Maria's colleague covering for her vacation, I want to...

11. ...open a property and read its `context.md` and recent audit log, and be productive within 5 minutes — without having to call Maria.
-

Core demo scenario (script)

1. **Voice call lands.** A "tenant" calls a LiveKit number; background noise (café audio) is playing through the speaker.
 2. **ai-coustics cleans the audio** in real time. Gadium (LiveKit voice agent) handles STT, asks one clarifying question ("Which unit?"), and produces a structured ticket payload.
 3. **Ticket appears in Maria's inbox** with `source: "voice"` within ~3 seconds of the call ending.
 4. **Maria gets a Slack ping:** " New ticket for Gartenstraße 42 — Review in Buena →".
 5. She clicks the deep link, arrives on the ticket detail page.
 6. She sees: the ticket, the **call transcript** with cleaned audio playback, a **proposed action**, the **context version** it was based on, the **cited sections** of that context, and the **policies** that were evaluated.
 7. She **approves, edits then approves, or rejects with a reason**.
 8. On approve, a new context version is created summarizing the outcome.
 9. She opens the **diff viewer** to see exactly what changed in the property's memory.
 10. She opens the **Suggestions Panel**, where Hermes has noticed that 4 of the last 10 heating tickets were approved under €200 — and recommends a new auto-approval policy. She accepts, creating a new context version with the policy attached.
-

4. Scope — What We Build

4.1 The Context Artifact

Each property has exactly **one active** `context.md`, with full version history.

Structure (convention, not enforced schema):

- YAML frontmatter: `property_id`, `version`, `updated_at`, `updated_by`

- Markdown body organized by `##` headings (e.g., `## Basics`, `## Tenants`, `## Known Issues`, `## Policies`, `## Source Documents`)

Versioning rules:

- Every write creates a new immutable row; no in-place edits.
- Version numbers are monotonic integers per property, starting at 1.
- Every version records `created_by` (operator email or `"system"` or `"hermes"`) and `created_reason` (free text).
- Hermes-suggested versions are flagged but **never auto-applied** — they always wait for human approval.

4.2 Features in the WebUI

Feature	Description	Priority
Property list	Sidebar with 2-3 seeded properties	P0
Context viewer	Rendered markdown with anchor links per heading	P0
Context editor	Monaco/CodeMirror editor; save creates new version	P0
Version history	List of versions with author, reason, timestamp	P0
Diff viewer	Unified diff between any two versions	P0
Ticket inbox	Table of tickets per property with status + source badge (voice/email/slack/manual)	P0
Ticket detail	Ticket body + proposal panel + citations + decision buttons. For voice tickets: transcript + audio playback	P0
Voice transcript viewer	Side-by-side: cleaned audio waveform + STT transcript with timestamps	P0
Policies page	CRUD over property policies (simple form)	P0
Sources page	List of allowlisted source documents (read-only for MVP)	P1
Audit log	Chronological list of all actions, filterable by property and actor (incl. <code>hermes</code>)	P0
Suggestions Panel	Hermes-generated suggestions with accept/dismiss buttons	P0
Simulate ticket panel	Dev-only: 4-5 templates + voice call trigger, one-click ticket creation	P0

4.3 Backend responsibilities

1. Persist properties, context versions, chunks, tickets, proposals, policies, sources, audit entries, **call sessions, transcripts, and Hermes suggestions**.
2. Expose a REST API for the WebUI.
3. Accept ticket ingestion via a single endpoint, **regardless of source channel**.
4. Run a **LiveKit-based voice ingestion server** that uses the ai-coustics SDK for noise suppression and produces a normalized ticket payload.
5. Trigger a **proposal generation pipeline** when a ticket is created.
6. Run an **async Hermes worker** that periodically analyzes recent decisions and emits suggestions.
7. Send an outbound Slack notification when a proposal is ready or a high-value Hermes suggestion appears.
8. Record every read/write to context, every decision on a proposal, and every Hermes output.

4.4 The "intelligence" pipeline — PLACEHOLDER

 This section is intentionally a placeholder. The developer will implement it by hand during the hackathon.

Interface contract the rest of the system depends on:

```
ts

interface ProposalEngine {
  generateProposal(input: {
    ticket: Ticket;
    property: Property;
    contextVersion: ContextVersion;
    policies: Policy[];
  }): Promise<{
    proposedAction: Record<string, unknown>;
    reasoning: string;
    citations: Array<{
      heading: string;
      excerpt: string;
      chunkId?: string;
    }>;
    confidence?: number;
  }>;
}
```

What the rest of the system assumes:

- The engine is **deterministic from the outside**: same ticket + same context + same policies
→ a proposal object that conforms to the interface above.
- The engine **does not mutate** any database state. It only reads inputs and returns an object.
- The caller (the orchestration layer) is responsible for persisting the returned proposal, evaluating policies against it, and triggering notifications.

Similarly, the **policy evaluator** is a placeholder:

```
ts

interface PolicyEvaluator {
  evaluate(
    proposedAction: Record<string, unknown>,
    policies: Policy[]
  ): {
    allowed: boolean;
    requiresHumanApproval: boolean;
    violations: string[];
  };
}
```

For the MVP, the WebUI treats **all proposals as requiring human approval**, so the evaluator's output is informational, not blocking.

4.5 The voice ingestion pipeline — NEW

🔔 *New in v2.0. Adds tracks: telli & ai-coustics — Voice AI in the Wild.*

High-level flow:

```
Tenant phone call
  ↓
LiveKit room (inbound SIP or WebRTC)
  ↓
ai-coustics SDK (real-time noise suppression + speech enhancement)
  ↓
LiveKit voice agent ("Gadium")
  - STT (Deepgram or equivalent)
  - LLM turn-taking + clarifying questions
  - TTS for system replies
  ↓
Structured ticket payload
  ↓
POST /tickets (source = "voice")
  ↓
[same downstream pipeline as any other ticket]
```

Interface contract:

```
ts

interface VoiceIngestionResult {
  callSessionId: string;           // FK into call_sessions
  ticketPayload: {
    property_id: string;           // resolved from caller phone number or asked
    raised_by: string;             // resolved from CRM lookup or asked
    subject: string;               // LLM-derived from transcript
    body: string;                  // full cleaned transcript
  };
  cleanedAudioUrl: string;         // post ai-coustics
  rawAudioUrl: string;             // pre ai-coustics, for A/B demo
  transcript: Array<{
    speaker: "tenant" | "agent";
    text: string;
    startMs: number;
    endMs: number;
  }>;
  noiseReductionDb: number;        // for the demo "wow" moment
}
```

Critical demo property: the system stores **both raw and cleaned audio** so we can A/B them on stage. Judges hearing the difference is the single most memorable moment of the demo.

4.6 The Hermes learning loop — NEW

🧠 *New in v2.0. Async, suggestion-only, never auto-applies.*

Hermes is a background worker, not part of the request path.

Cadence: runs every N minutes (or on-demand via `/dev/run-hermes`).

Inputs:

- Last K tickets per property (default K=20)
- Their proposals, decisions (approve/edit/reject), and rejection reasons
- Current context version
- Current policy set

Outputs (one or more `hermes_suggestions` rows per run):

```
ts
```

```

type HermesSuggestion =
  | {
    kind: "policy_suggestion";
    property_id: string;
    message: string; // human-readable
    proposed_policy: Partial<Policy>; // a draft policy row
    evidence: Array<{ ticket_id: string; reason: string }>;
  }
  | {
    kind: "context_addition";
    property_id: string;
    message: string;
    proposed_section_heading: string;
    proposed_section_md: string;
    evidence: Array<{ ticket_id: string; reason: string }>;
  }
  | {
    kind: "context_correction";
    property_id: string;
    message: string;
    target_heading: string;
    proposed_replacement_md: string;
    evidence: Array<{ ticket_id: string; reason: string }>;
  };

```

Hard rules (these are sacred):

1. Hermes **never writes to** `context_versions`. It only writes to `hermes_suggestions`.
2. Hermes **never modifies policies**. It only proposes them.
3. Every Hermes suggestion carries **evidence** (ticket IDs + reasons). No suggestion without justification.
4. Hermes runs are **logged in the audit log** with `actor = "hermes"`.

5. Data Model

All identifiers are UUIDs unless specified. Timestamps are `timestamptz`.

5.1 `properties`

Field	Type	Notes
<code>id</code>	uuid PK	

Field	Type	Notes
<code>name</code>	text	e.g., "Gartenstraße 42, Berlin"
<code>slack_channel</code>	text nullable	target channel for notifications
<code>phone_number</code>	text nullable	inbound number routed to LiveKit; used to resolve property on incoming call
<code>created_at</code>	timestampz	

5.2 `context_versions`

Field	Type	Notes
<code>id</code>	uuid PK	
<code>property_id</code>	uuid FK → properties	
<code>version</code>	int	monotonic per property
<code>content_md</code>	text	full markdown
<code>frontmatter</code>	jsonb	parsed yaml
<code>created_by</code>	text	operator email, <code>"system"</code> , or <code>"hermes"</code>
<code>created_reason</code>	text	free text
<code>parent_version</code>	int nullable	
<code>source_suggestion_id</code>	uuid nullable	FK → <code>hermes_suggestions</code> if originated there
<code>created_at</code>	timestampz	

Unique constraint on `(property_id, version)`.

5.3 `context_chunks`

Field	Type	Notes
<code>id</code>	uuid PK	
<code>context_version_id</code>	uuid FK	
<code>property_id</code>	uuid FK	denormalized for query speed
<code>heading</code>	text	the <code>##</code> heading this chunk belongs to

Field	Type	Notes
<code>content</code>	text	the chunk body
<code>embedding</code>	vector(1536) nullable	populated by the placeholder pipeline

5.4 `property_policies`

Field	Type	Notes
<code>id</code>	uuid PK	
<code>property_id</code>	uuid FK	
<code>kind</code>	text	e.g., <code>"approval_threshold"</code> , <code>"tone"</code> , <code>"forbidden"</code>
<code>rule_json</code>	jsonb	free-form
<code>description</code>	text	human-readable summary
<code>active</code>	boolean	
<code>created_at</code>	timestamptz	

5.5 `context_sources`

Field	Type	Notes
<code>id</code>	uuid PK	
<code>property_id</code>	uuid FK	
<code>kind</code>	text	<code>"pdf"</code> , <code>"email"</code> , <code>"erp"</code> , <code>"manual"</code> , <code>"voice"</code>
<code>uri</code>	text	
<code>sha256</code>	text	
<code>status</code>	text	<code>"allowed"</code> , <code>"pending"</code> , <code>"rejected"</code>
<code>approved_by</code>	text nullable	
<code>approved_at</code>	timestamptz nullable	

5.6 tickets

Field	Type	Notes
id	uuid PK	
property_id	uuid FK	
source	text	"slack", "email", "api", "simulated", "voice"
external_id	text nullable	
raised_by	text	e.g., "Herr Schmidt, Unit 3B"
subject	text	
body	text	
call_session_id	uuid nullable	FK → call_sessions if source = "voice"
status	text	"open", "proposed", "approved", "rejected", "resolved"
created_at	timestampz	

5.7 call_sessions — NEW

Field	Type	Notes
id	uuid PK	
property_id	uuid FK	resolved on call accept
caller_phone	text	E.164
started_at	timestampz	
ended_at	timestampz nullable	
duration_ms	int nullable	
livekit_room	text	for debugging
raw_audio_url	text	pre ai-coustics
cleaned_audio_url	text	post ai-coustics

Field	Type	Notes
<code>transcript_jsonb</code>	jsonb	array of turn objects
<code>noise_reduction_db</code>	numeric nullable	average dB reduction; for demo bragging rights
<code>agent_summary</code>	text	one-line summary produced by the voice agent

5.8 `agent_proposals`

Field	Type	Notes
<code>id</code>	uuid PK	
<code>ticket_id</code>	uuid FK	
<code>context_version_id</code>	uuid FK	required — the exact snapshot used
<code>proposed_action</code>	jsonb	from the engine placeholder
<code>reasoning</code>	text	
<code>citations</code>	jsonb	array of <code>{heading, excerpt, chunk_id?}</code>
<code>policy_evaluation</code>	jsonb nullable	output of the evaluator placeholder
<code>status</code>	text	<code>"pending"</code> , <code>"approved"</code> , <code>"edited"</code> , <code>"rejected"</code>
<code>reviewed_by</code>	text nullable	
<code>reviewed_at</code>	timestampz nullable	
<code>final_action</code>	jsonb nullable	what was actually executed
<code>rejection_reason</code>	text nullable	
<code>slack_message_ts</code>	text nullable	for future update-in-place
<code>created_at</code>	timestampz	

5.9 `hermes_suggestions` — NEW

Field	Type	Notes
<code>id</code>	uuid PK	

Field	Type	Notes
<code>property_id</code>	uuid FK	
<code>kind</code>	text	<code>"policy_suggestion"</code> , <code>"context_addition"</code> , <code>"context_correction"</code>
<code>message</code>	text	human-readable
<code>payload_jsonb</code>	jsonb	<code>proposed_policy</code> or <code>proposed_section_md</code> etc.
<code>evidence_jsonb</code>	jsonb	<code>[{ ticket_id, reason }, ...]</code>
<code>status</code>	text	<code>"pending"</code> , <code>"accepted"</code> , <code>"dismissed"</code>
<code>reviewed_by</code>	text nullable	
<code>reviewed_at</code>	timestampz nullable	
<code>applied_context_version_id</code>	uuid nullable	populated when accepted and merged
<code>applied_policy_id</code>	uuid nullable	populated when accepted as a policy
<code>created_at</code>	timestampz	

5.10 `audit_log`

Field	Type	Notes
<code>id</code>	uuid PK	
<code>actor</code>	text	operator email, <code>"system"</code> , or <code>"hermes"</code>
<code>action</code>	text	e.g., <code>"context.read"</code> , <code>"context.write"</code> , <code>"proposal.approve"</code> , <code>"proposal.reject"</code> , <code>"policy.update"</code> , <code>"ticket.create"</code> , <code>"call.start"</code> , <code>"call.end"</code> , <code>"hermes.run"</code> , <code>"hermes.suggest"</code> , <code>"hermes.accept"</code> , <code>"hermes.dismiss"</code>
<code>property_id</code>	uuid nullable	
<code>entity_type</code>	text	

Field	Type	Notes
<code>entity_id</code>	uuid nullable	
<code>metadata</code>	jsonb	
<code>created_at</code>	timestampz	

6. REST API

All responses are JSON. All errors return `{ error: string, details?: unknown }` with appropriate HTTP status. Base path: `/api/v1`.

6.1 Properties

Method	Path	Purpose
<code>GET</code>	<code>/properties</code>	List all properties
<code>GET</code>	<code>/properties/:id</code>	Get one property

6.2 Context

Method	Path	Purpose
<code>GET</code>	<code>/properties/:id/context</code>	Get current (latest) context version
<code>GET</code>	<code>/properties/:id/context/versions</code>	List all versions (metadata only)
<code>GET</code>	<code>/properties/:id/context/versions/:version</code>	Get a specific version with full content
<code>POST</code>	<code>/properties/:id/context</code>	Create a new version. Body: <code>{ content_md, reason, created_by, source_suggestion_id? }</code>
<code>GET</code>	<code>/properties/:id/context/diff?from=X&to=Y</code>	Return unified diff between two versions

6.3 Policies

Method	Path	Purpose
GET	/properties/:id/policies	List policies
POST	/properties/:id/policies	Create policy
PATCH	/policies/:policyId	Update policy
DELETE	/policies/:policyId	Delete policy

6.4 Sources

Method	Path	Purpose
GET	/properties/:id/sources	List allowlisted sources
POST	/properties/:id/sources	Register a source (MVP: manual entry)
PATCH	/sources/:sourceId	Change status (approve/reject)

6.5 Tickets

Method	Path	Purpose
GET	/properties/:id/tickets	List tickets
GET	/tickets/:id	Get ticket + its latest proposal + call session if voice
POST	/tickets	Create ticket. Body: { property_id, source, raised_by, subject, body, call_session_id? }. Triggers the proposal pipeline asynchronously.

6.6 Proposals

Method	Path	Purpose
GET	/tickets/:ticketId/proposals	List proposals for a ticket (usually one)
GET	/proposals/:id	Get proposal detail
POST	/proposals/:id/approve	Approve. Body: { reviewed_by, final_action?, context_update_md? }

Method	Path	Purpose
POST	/proposals/:id/edit	Edit then approve. Body: { reviewed_by, final_action, context_update_md? }
POST	/proposals/:id/reject	Reject. Body: { reviewed_by, rejection_reason }

6.7 Voice — NEW

Method	Path	Purpose
GET	/call-sessions/:id	Get full call session: transcript, both audio URLs, summary
GET	/properties/:id/call-sessions	List recent call sessions for a property
POST	/dev/simulate-call	Trigger a synthetic voice call from a fixture audio file. Body: { property_id, fixture_id }

Inbound call handling itself is not REST — it is driven by LiveKit's webhook + agent runtime. The voice agent calls `POST /tickets` once it has a structured payload.

6.8 Hermes — NEW

Method	Path	Purpose
GET	/properties/:id/suggestions?status=pending	List Hermes suggestions
POST	/suggestions/:id/accept	Accept a suggestion. Body: { reviewed_by, edits? }. Creates a new context version or policy as appropriate.
POST	/suggestions/:id/dismiss	Dismiss. Body: { reviewed_by, reason? }
POST	/dev/run-hermes	Force a Hermes run now. Body: { property_id? }

6.9 Audit

Method	Path	Purpose
GET	/audit?property_id=X&actor=Y&limit=N	Paginated audit entries

6.10 Dev / Demo

Method	Path	Purpose
POST	/dev/simulate-ticket	Create a ticket from a template
POST	/dev/simulate-call	Trigger a synthetic voice call (see §6.7)
POST	/dev/run-hermes	Force a Hermes run (see §6.8)
POST	/dev/reset	Reseed the database

7. System Flows

7.1 Ticket ingestion → proposal (channel-agnostic)

1. Client calls `POST /tickets` (from email parser, Slack handler, voice agent, simulator, or manual UI).
2. Backend validates, persists ticket with `status = "open"`.
3. Backend emits `audit: ticket.create` (with `source` in metadata).
4. Backend enqueues a job to the **proposal pipeline placeholder**.
5. Pipeline placeholder:
 - Loads current context version.
 - Loads active policies.
 - Calls `ProposalEngine.generateProposal(...)` — placeholder.
 - Calls `PolicyEvaluator.evaluate(...)` — placeholder.
 - Persists an `agent_proposals` row with `status = "pending"`.
 - Updates ticket to `status = "proposed"`.
 - Emits `audit: proposal.create` and `audit: context.read`.
 - Sends Slack notification.
6. WebUI surfaces the new proposal (polling or SSE).

7.2 Voice ticket ingestion — NEW

1. Inbound call arrives at LiveKit (SIP trunk or test WebRTC client).
2. LiveKit webhook fires; we accept the call, create a `call_sessions` row (`started_at` set), emit `audit: call.start`.
3. ai-coustics SDK is wired into the audio track (real-time noise suppression, speech enhancement).
4. The voice agent ("Gadium"):
 - Greets the caller in German.
 - Resolves the property by `caller_phone` lookup (fallback: ask).
 - Captures the issue, asks at most one clarifying question.
 - Generates a one-line summary + structured payload.
5. On call end:
 - Persist `cleaned_audio_url`, `raw_audio_url`, `transcript_jsonb`, `noise_reduction_db`, `agent_summary`, `ended_at`, `duration_ms`.
 - Emit `audit: call.end`.
 - Call `POST /tickets` with `source = "voice"` and `call_session_id`.
6. From step 6 onward, identical to §7.1.

7.3 Operator decision

- **Approve:** proposal status → `"approved"`, ticket status → `"approved"`, optionally create new context version, emit audit entries.
- **Edit then approve:** same, but `final_action` differs from `proposed_action`.
- **Reject:** proposal status → `"rejected"`, ticket status → `"rejected"`, store reason, emit audit.

In all cases, emit `audit: proposal.<decision>`.

7.4 Context edit (outside a ticket)

Operator edits `context.md` directly in the WebUI. Save → `POST /properties/:id/context` → new version, audit entry `context.write`.

7.5 Policy change

Operator adds/edits a policy. On the next ticket, the pipeline loads the new policy set. No recomputation of past proposals.

7.6 Hermes learning loop — NEW

1. Worker fires (cron or `/dev/run-hermes`).

2. For each property:
 - Load last K tickets + proposals + decisions.
 - Run pattern detection (placeholder logic — implemented by hand).
 - For each detected pattern, write a `hermes_suggestions` row with `status = "pending"`, evidence, and a draft payload.
 - Emit `audit: hermes.run` and `audit: hermes.suggest`.
3. If a suggestion crosses a "high-value" threshold (configurable), emit a Slack ping.
4. Operator opens the **Suggestions Panel** in the WebUI.
5. **Accept:** depending on `kind`:
 - `policy_suggestion` → create row in `property_policies`; suggestion status → `"accepted"`; populate `applied_policy_id`.
 - `context_addition` / `context_correction` → create new `context_versions` row with `created_by = operator_email`, `source_suggestion_id = suggestion.id`; suggestion status → `"accepted"`; populate `applied_context_version_id`.
 - Emit `audit: hermes.accept` + the corresponding `policy.create` / `context.write`.
6. **Dismiss:** suggestion status → `"dismissed"`, store reason, emit `audit: hermes.dismiss`.

7.7 Architectural rules (do not break these)

1. **Context is sacred.** Only human-approved writes ever touch `context_versions`. Hermes proposes; humans dispose.
 2. **Stateless except Hermes.** The proposal engine never holds state across calls. Hermes is the only learning component, and it learns by reading the audit trail.
 3. **Every step is auditable.** Input → proposal → decision → context → learning. There are no silent mutations.
 4. **Async > realtime for intelligence.** Proposals are fast (request-path). Learning is background. The user experience never blocks on Hermes.
-

8. Slack Notification (Outbound-Only)

- One incoming webhook URL per channel (configured via env var or `properties.slack_channel`).
- Payload: simple text + link. No blocks, no buttons.
- Triggers:
 - Proposal created
 - Proposal resolved (approved or rejected)

- **High-value Hermes suggestion appears** (NEW)
- **Voice call completed and ticket created** (NEW — optional, useful for the demo)

Examples:

 New ticket for Gartenstraße 42 ( voice call)
From: Herr Schmidt (Unit 3B)
Subject: Heizung kalt
→ Review in Buena: <<https://buena.local/tickets/abc-123>>

 Hermes has a suggestion for Gartenstraße 42
"Heating issues under €200 were approved 4 times this week."
→ Review in Buena: <<https://buena.local/properties/.../suggestions/xyz>>

If Slack is not configured, the notifier silently logs to console. **The demo must not break if Slack is unreachable.**

9. Seed Data Requirements

For the demo to be credible, the seed script must create:

- **2 properties** with realistic German addresses, each with a `phone_number` for voice routing.
 - For each property: a carefully hand-written `context.md` (v1) covering basics, 2–3 tenants, 1–2 known issues, 2–3 policies, 1–2 source document references.
 - **3–4 active policies** per property, including at least one `approval_threshold` rule.
 - **1–2 closed tickets** with historical proposals already approved (to populate version history and audit log).
 - **Historical pattern data:** at least 4 closed tickets matching the pattern Hermes will "discover" during the demo (e.g., 4 heating tickets all approved under €200), so the suggestion is grounded in real evidence.
 - **4–5 ticket templates** for the "Simulate Ticket" panel: heating issue, noise complaint, repair request, lease question, payment delay.
 - **2 voice fixtures** (pre-recorded audio files): one clean, one with heavy background noise (café/street). Used by `/dev/simulate-call` to demo ai-coustics on stage without depending on a live phone.
-

10. Tech Stack

Layer	Choice
Monorepo	pnpm workspaces
Backend	Node.js + TypeScript + Fastify
ORM	Prisma
Database	PostgreSQL 16 + <code>pgvector</code>
Queue	BullMQ + Redis (or synchronous fallback)
Frontend	Next.js 14 (App Router) + Tailwind + shadcn/ui
Shared types	zod schemas in <code>packages/shared</code>
Markdown	<code>react-markdown</code> , <code>remark-gfm</code>
Editor	Monaco or CodeMirror 6
Diff	<code>diff</code> (node) + <code>react-diff-viewer-continued</code>
Realtime	Server-Sent Events
Voice runtime	LiveKit Agents (Node SDK)
Audio cleanup	ai-coustics SDK (in-line on the audio track)
STT	Deepgram (or LiveKit-integrated provider)
TTS	ElevenLabs or LiveKit default
Audio waveform UI	wavesurfer.js
Container	Docker Compose (Postgres + Redis + LiveKit dev server)

11. Repository Layout

```
buena-context-engine/
├─ apps/
│   └─ api/
│       └─ src/
│           └─ routes/           # tickets, properties, context, policies,
# proposals, audit, dev, voice, suggestions
```

```

├── services/
│   ├── context/    # versioning, diff
│   ├── tickets/
│   ├── proposals/  # orchestration (calls placeholder engine)
│   ├── engine/     # 🧩 PLACEHOLDER – implemented by hand
│   ├── policies/   # 🧩 PLACEHOLDER evaluator
│   ├── hermes/     # 🧠 NEW – async learning worker
│   ├── notifier/   # Slack webhook
│   └── audit/
├── db/              # Prisma schema + migrations + seed
└── index.ts

├── voice/           # 🗣️ NEW – LiveKit agent process
│   └── src/
│       ├── agent.ts    # Gradium voice agent
│       ├── aicoustics.ts # SDK wiring
│       └── ingest.ts    # POSTs to /tickets
└── web/
    └── src/app/
        ├── properties/
        ├── properties/[id]/context/
        ├── properties/[id]/tickets/[ticketId]/
        ├── properties/[id]/policies/
        ├── properties/[id]/sources/
        ├── properties/[id]/suggestions/ # 🧠 NEW
        ├── properties/[id]/calls/       # 🗣️ NEW
        └── audit/
├── packages/
│   └── shared/          # zod schemas, TS types
├── fixtures/           # 🗣️ NEW – sample audio files for /dev/simulate-call
├── docker-compose.yml
└── package.json

```

12. Build Plan & Milestones

Milestone 0 — Foundation (first 2 hours)

- Repo + pnpm workspaces
- Docker Compose with Postgres + Redis (+ LiveKit dev mode if available)
- Prisma schema per §5 (including new tables)
- Seed script
- `packages/shared` with zod schemas for all API payloads
- `/health` endpoint

- Next.js app renders an empty sidebar

Exit criteria: `pnpm dev` brings up API + Web + DB, seed runs, one property visible in sidebar.

Milestone 1 — Context plane (hours 2-6)

- All `/context` endpoints functional
- Versioning service with monotonic versions
- Diff service
- Context viewer, editor, version list, diff viewer in WebUI
- Audit entries for every read/write

Exit criteria: operator can view, edit, save, and diff context through the WebUI.

Milestone 2 — Ticket plane (hours 6-10)

- `POST /tickets` ingestion (channel-agnostic)
- "Simulate Ticket" panel with templates
- Ticket inbox (with source badges) and detail pages
- Proposal stored with `context_version_id` reference
- Approve / Edit / Reject endpoints
- On approve, optionally create new context version

Exit criteria: end-to-end flow with a hardcoded stub proposal. Entire UI works.

Milestone 3 — Voice plane (hours 10-14) — NEW

- LiveKit agent process scaffolded
- ai-coustics SDK integrated; raw + cleaned audio both saved
- Deepgram STT producing transcript turns
- Agent collects subject + body + property resolution
- `POST /tickets` called on call end with `source = "voice"`
- `/dev/simulate-call` works against a fixture file (no live phone needed)
- Ticket detail page renders transcript + waveform + audio playback (raw vs cleaned A/B toggle)

Exit criteria: `/dev/simulate-call` with the noisy fixture produces a clean transcript and a normal-looking ticket. A/B toggle is the demo's "wow" moment.

Milestone 4 — Hermes learning loop (hours 14-17) — NEW

- `hermes_suggestions` table + endpoints
- Async worker (BullMQ job) that runs on `/dev/run-hermes`

- Pattern-detection placeholder (implemented by hand) — at minimum, detects "N approvals of similar tickets under threshold X"
- Suggestions Panel UI with accept/dismiss
- Accepting a suggestion creates a new context version or policy
- Audit entries for every Hermes action

Exit criteria: click "Run Hermes" → suggestion appears → accept → new context version with `created_by = operator, source_suggestion_id = ...`.

Milestone 5 — Intelligence placeholder (hours 17-22)

- 🧩 Real `ProposalEngine` and `PolicyEvaluator` implemented by hand
- Real proposals replace stubs; demo story works end-to-end

Exit criteria: demo runs with real proposals, real citations, real Hermes evidence.

Milestone 6 — Polish & demo prep (hours 22-28)

- Slack notification wired with real webhook
- Audit log UI, filterable by actor (incl. `hermes`)
- Policies UI
- Sources UI (read-only)
- Realtime updates (SSE)
- Seed data polished with high-quality German content
- Demo script rehearsed 3 times with the noisy audio fixture
- A/B audio toggle tested on the actual demo machine's speakers

Exit criteria: 5-minute demo runs cleanly without a single manual page refresh, on the actual hackathon WiFi.

13. Demo Script (Target: 5 minutes)

Time	Action	What judges see
0:00	Show empty <code>#property-gartenstrasse</code> + WebUI dashboard	Clean, professional UI
0:15	Click "Simulate Call" with the noisy fixture	Audio plays briefly with café noise
0:25	Toggle A/B: raw vs ai-coustics-cleaned	Audible "wow" moment — judges hear the difference

Time	Action	What judges see
0:40	Voice ticket lands in inbox with 📞 badge	Ticket card animates in
0:50	Slack notification arrives	Credibility
1:00	Click deep link → ticket detail page	Transcript + proposal + reasoning + citations
1:20	Hover over citation → highlights section in context	Grounding is visible
1:40	Show the exact <code>context v11</code> that was used	Trust by construction
2:00	Click "Policies" → show approval threshold rule	Human control layer
2:20	Edit policy: lower threshold from €500 to €200	Change saved, audited
2:40	Trigger a second, similar ticket (this time email)	Same pipeline, different source
3:10	Approve it, write a context note	New version created
3:30	Open diff viewer: v11 → v12	Literal git-style diff
3:50	Open Suggestions Panel — Hermes proposes auto-approval policy with evidence	The "it gets smarter" moment
4:15	Accept the suggestion → new policy created, audit shows <code>hermes.accept</code>	Closed loop
4:35	Open audit log, filter by actor <code>hermes</code>	Full trail
4:55	Close with the four claims restated	Mic drop

14. Risks & Mitigations

Risk	Mitigation
Intelligence placeholder takes longer than expected	Entire system already works with stub proposals; demo is still shippable without the real pipeline
Live phone call fails on demo day	<code>/dev/simulate-call</code> against a fixture file is the primary demo path; live phone is a stretch
ai-coustics SDK integration is harder than expected	Fallback: pre-process the fixture file offline with the SDK, ship cleaned + raw files in the repo, keep the A/B toggle. The demo story is unchanged.

Risk	Mitigation
LiveKit dev server flaky	Run the agent against a recorded audio fixture; LiveKit not strictly required for the demo
Hermes pattern detection too naive to look smart	Hand-craft the seed data so the pattern is obvious; the suggestion text is template-driven
Slack webhook fails on demo day	Notifier degrades silently
Database state gets corrupted during rehearsals	<code>POST /dev/reset</code> reseeds in < 2 seconds
Realtime updates flaky	Fallback: 2-second polling on the inbox and ticket detail pages
Live LLM latency during demo	Pre-generate proposals for the exact demo tickets, cache them
Hackathon WiFi terrible	All AI calls have local fallback paths or pre-cached responses

15. Out of Scope (Post-MVP, but Worth Mentioning in Q&A)

- Real source connectors (email IMAP, ERP APIs, PDF watchers)
- Automatic context generation from source documents
- Multi-operator collaboration with conflict resolution
- Owner-facing read-only views of their property's context
- Per-tenant granular privacy controls
- Mobile app / PWA
- Outbound voice agents (the system makes calls, not just receives)
- Hermes auto-applying low-risk suggestions with a "trust score"
- Analytics on proposal acceptance rates, time-to-resolution, context freshness, Hermes accept-rate per kind

16. Open Questions for the Team

1. **SSE or polling?** *Recommendation: polling for MVP, SSE if time allows.*
2. **Where does "Simulate Ticket" / "Simulate Call" live?** *Recommendation: visible in demo mode, gated behind an env flag in production.*

3. **Do we show the placeholder engine's intermediate reasoning in the UI?**

Recommendation: citations only for MVP; add a "debug drawer" if time allows.

4. **Live phone vs fixture for the demo?** *Recommendation: fixture is the primary path. If live phone works in dress rehearsal, use it as the opener; if not, no one knows.*

5. **Should Hermes run continuously or only on-demand during the hackathon?**

Recommendation: on-demand via `/dev/run-hermes` for predictable demos; expose the "scheduled" framing in the pitch.

17. Pitch Positioning (one paragraph for the judges)

"Most AI systems hallucinate company reality at runtime. We don't. Every property at Buena gets a living, versioned `context.md` — a single source of truth that AI agents read before acting and that humans curate after. Tenant inputs come from anywhere — email, Slack, or a noisy phone call cleaned in real time by ai-coustics — and they all land in the same auditable pipeline. No proposal is executed without human approval, no context is rewritten without a human in the loop, and every approved decision feeds an async learning agent that surfaces patterns as suggestions — never as silent mutations. We don't let AI act blindly. We give it memory, rules, and human oversight — and the system gets smarter, but only with human approval."

End of PRD v2.0.

The two placeholders — `ProposalEngine` and `PolicyEvaluator` — and the two newly added building blocks — the **voice ingestion pipeline** and the **Hermes learning loop** — are the parts of this document that the team will implement by hand during the hackathon. Everything else (database, APIs, WebUI, audit, versioning, diff, Slack) can be built in parallel against stable interfaces, so no one is blocked on how the intelligence layers are ultimately implemented.